Data Science

Decision Trees with CART



Advanced Modeling

GENERAL ASSEMBLY

Lesson roadmap

Welcome + Introduction

Decision Tree Models

The Decision Tree Algorithm

Representations

Splitting Criteria

Bring It Home

Learning objectives



1. Understand the intuition behind decision trees.
2. Understand the CART algorithm that powers decision trees.
3. Describe how decision trees use Gini to make decisions.
4. Fit, generate predictions from, and evaluate decision tree models.
5. Tune `max_depth`, `min_samples_split`, `min_samples_leaf`, to get the best possible tree.
6. Visualize a Decision Tree.

Decision Tree Models

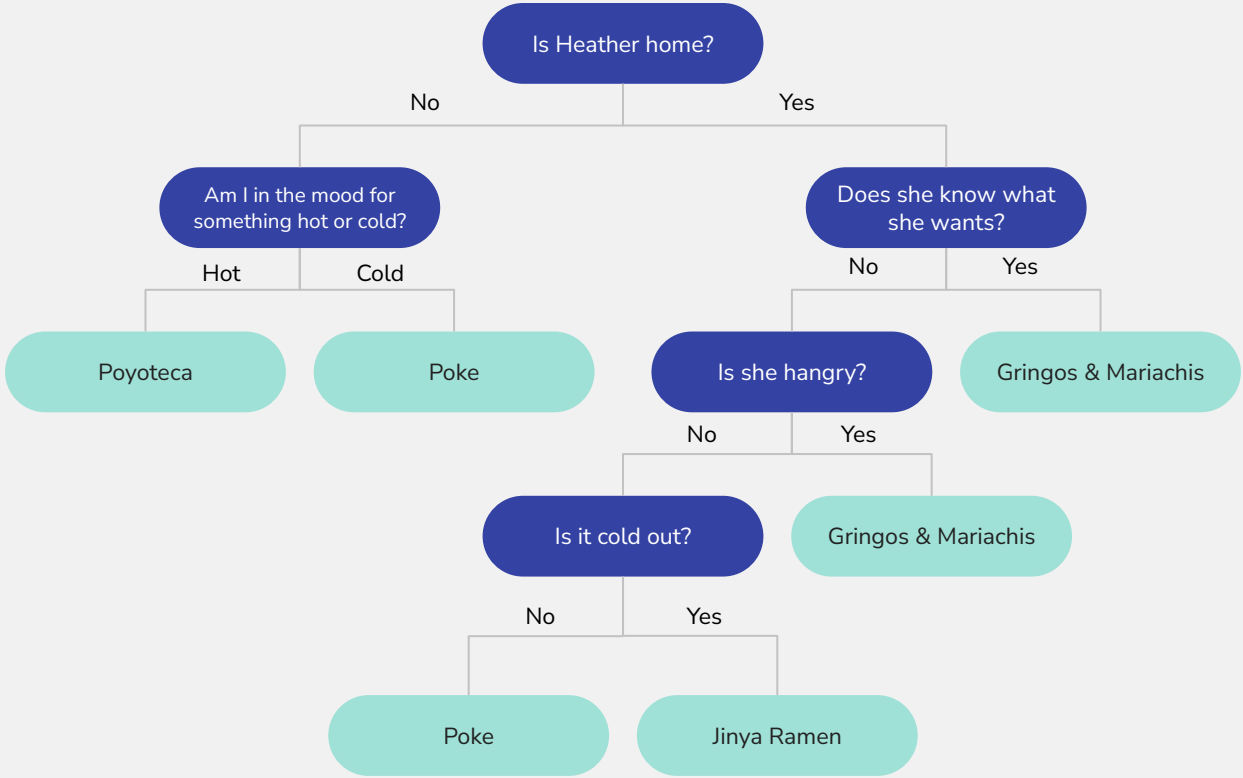
Decision Trees

We make decisions every day, often without even thinking about it.

A **decision tree** is basically a flowchart that terminates with a decision. Each split in the flowchart represents some condition being true or false.

 © 2025 General Assembly

What am I going to eat for dinner?



 © 2025 General Assembly

Decision Tree Models

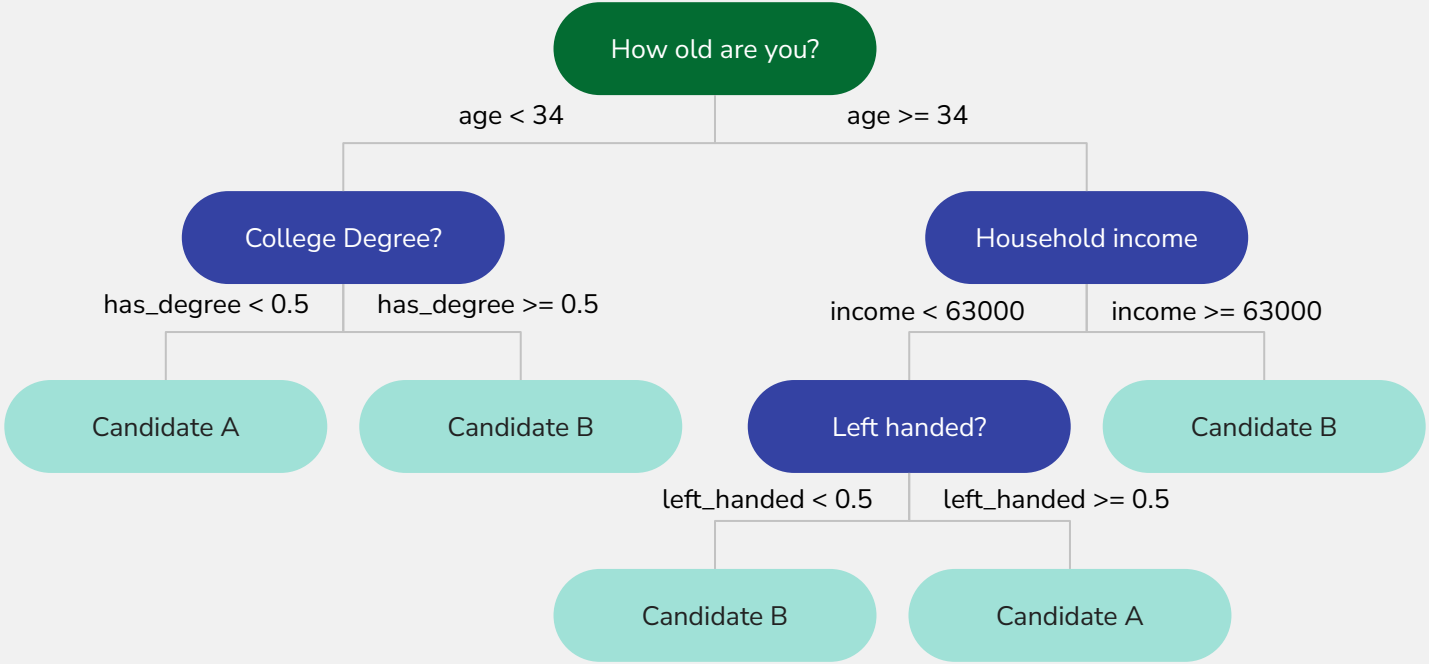
We'll be fitting decision trees as models, and so **their structure will be learned from our data**.

For linear models, an algorithm learns β from our data.

For tree-based models, an algorithm learns **when/where tree splits occur**.

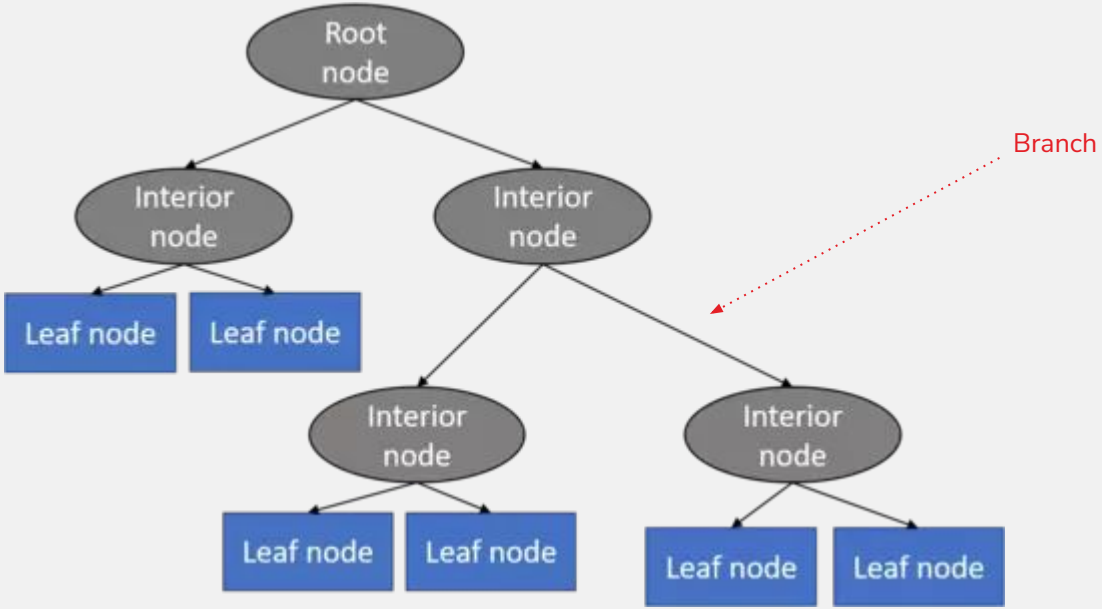
 © 2025 General Assembly

Example: Who will a given person vote for?



 © 2025 General Assembly

Some terminology: nodes, leaves and branches



 © 2025 General Assembly

Source

The Decision Tree CART Algorithm

The CART Algorithm

So how is this sort of model actually fit? Most commonly, the **CART (Classification and Regression Trees) algorithm** is used.

Let’s take a look at how it works on the next slide.

 © 2025 General Assembly

The CART Algorithm

1. Find the "best" possible split value in each variable, across each variable based on some **splitting criterion**.
2. Split your data at this optimal split.
3. Repeat steps 1-2 for each new subset of the data you've created until some **stopping criterion** is met.
4. *(There's an optional **pruning step**, but not by default!)*

 © 2025 General Assembly

When do we stop?

The **stopping criterion** for a decision tree is actually up to you! It can be considered a **tuning parameter** as well, actually.

By default, a decision tree will not stop splitting until each observation is in its own leaf! This is the perfectly overfit model, so **any imposed stopping criterion can be thought of as a regularizing factor**.

 © 2025 General Assembly

Stopping criteria allowed in Scikit-Learn

- `max_depth`: How many levels deep will we allow our tree to grow?
- `min_samples_split`: What is the minimum number of samples needed to split a node? If a split is not possible based on this criterion, we're done.
- `min_samples_leaf`: What is the minimum number of samples that we allow in a leaf node?
  - This is so similar to `min_samples_split` that we usually don't touch this.
- `min_impurity_decrease`: What's the minimum amount we need our stopping criterion to drop to allow a split? (We won't touch this one)

 © 2025 General Assembly

Ways to represent a tree

“Partitioning the Feature Space”

Decision trees are pretty and easy to read. For that we consider decision trees to be **white box models**.

However, there’s another representation of decision trees that we sometimes prefer to consider. We can think of these splits as:

“partitioning the feature space”

 © 2025 General Assembly

“Partitioning the Feature Space”

Decision trees are pretty and easy to read. For that we consider decision trees to be **white box models**.

However, there’s another representation of decision trees that we sometimes prefer to consider. We can think of these splits as:

“partitioning the feature space”

separating into many parts

the range of all values our *xs* take

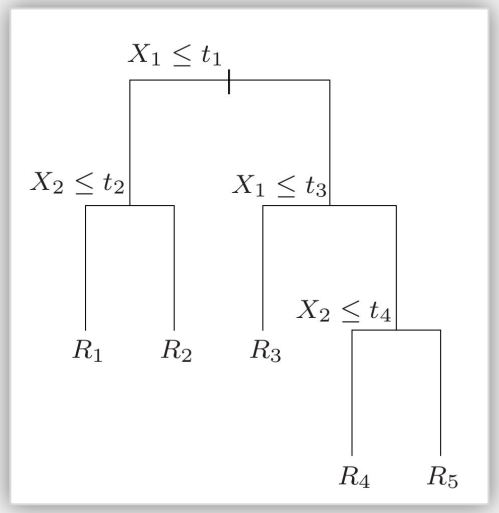
 © 2025 General Assembly

Let’s draw a picture

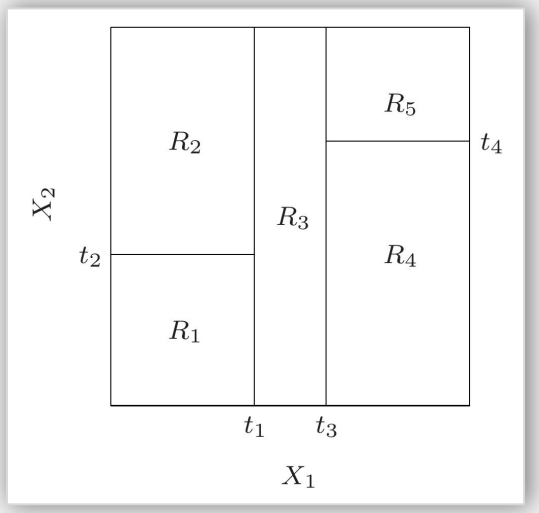


Source

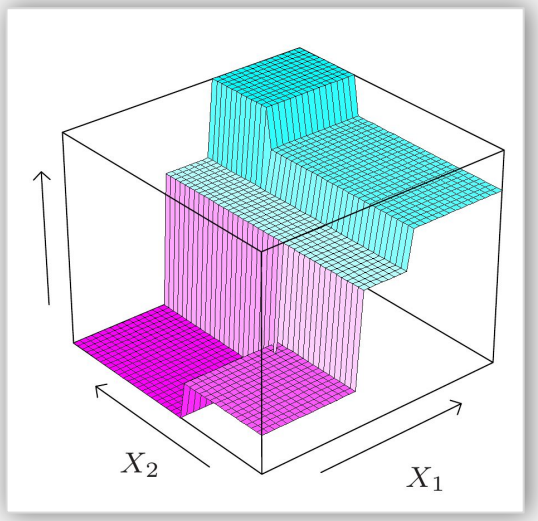
Three ways to draw the same thing!



Decision tree's tree representation



Decision tree's partitioning of the feature space representation



Same as the second image drawn in 3D for a regression problem

 © 2025 General Assembly

Source: Elements of Statistical Learning (ESL), by Hastie, Tibshirani, and Friedman

Splitting Criteria

Splitting Criteria

What makes one split better than another? The **splitting criteria** is the one that minimizes some **loss function** the most if the model were to stop right there.

For regression problems, this loss function is simply the **mean squared error (MSE)**.

For classification, you might think we'd use accuracy or something, but in fact, we're going to learn a new one.

 © 2025 General Assembly

The Gini Impurity Index

The Gini index is a loss function used to describe the **purity** of the result of a classification problem.

Let p_i be the proportion of class *i* in a given leaf node. The **Gini impurity index** of that node is:

$$G = 1 - \sum p_i^2$$

 © 2025 General Assembly

The Gini Impurity Index Examples

$$G = 1 - \sum p_i^2$$

Consider a binary classification problem. In a **pure** node (i.e., one that's totally one class), we have

$$G = 1 - \sum p_i^2 = 1 - 0^2 - 1^2 = 0$$

And in perfectly **impure** node where the classes are 50/50, we have

$$G = 1 - \sum p_i^2 = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 1 - \frac{1}{4} - \frac{1}{4} = \frac{1}{2}$$

 © 2025 General Assembly

Let's make it real

We are predicting whether or not someone will buy an **iPhone** or an **Android**:

- At a particular node, there are **25 observations** (phone buyers) of whom **10 bought iPhones and 15 bought Androids**.

Let's calculate the Gini index before making any split:

$$1-\left(\frac{iPhone}{Total}\right)^2-\left(\frac{Android}{Total}\right)^2=1-\left(\frac{10}{25}\right)^2-\left(\frac{15}{25}\right)^2=0.48$$

- The **maximum value** of the Gini index is 0.5 and occurs when the classes are perfectly balanced in a node.
- The **minimum value** of the Gini index is 0 and occurs when there is only one class represented in a node.
- A node with a lower Gini index is said to be more "pure."

 © 2025 General Assembly

Now let's split on either gender or age

Gender

- **Males:** Two iPhones and 12 Androids, thus the predicted class is **Android**.
- **Females:** Eight iPhones and three Androids, thus the predicted class is **iPhone**.

Age

- **30 or younger:** Four iPhones and eight Androids, thus the predicted class is **Android**.
- **31 or older:** Six iPhones and seven Androids, thus the predicted class is **Android**.

 © 2025 General Assembly

Evaluating the split on gender using the Gini index

$$\textbf{Males:} 1 - \left(\frac{2}{14}\right)^2 - \left(\frac{12}{14}\right)^2 = 0.24$$

$$\textbf{Females:} 1 - \left(\frac{8}{11}\right)^2 - \left(\frac{3}{11}\right)^2 = 0.40$$

$$\textbf{Weighted Average:} 0.24\left(\frac{14}{25}\right) + 0.40\left(\frac{11}{25}\right) = 0.31$$

 © 2025 General Assembly

Evaluating the split on age using the Gini index

$$\textbf{30 or younger:} 1 - \left(\frac{4}{12}\right)^2 - \left(\frac{8}{12}\right)^2 = 0.44$$

$$\textbf{31 or older:} 1 - \left(\frac{6}{13}\right)^2 - \left(\frac{7}{13}\right)^2 = 0.50$$

$$\textbf{Weighted Average:} 0.44\left(\frac{12}{25}\right) + 0.50\left(\frac{13}{25}\right) = 0.47$$

 © 2025 General Assembly

What does it all mean?

The decision tree algorithm will try every possible split and will choose the one that reduces the Gini index (and thus increases the "node purity") the most.

You can think of this as each split increasing the accuracy of predictions. If there is some error at a node, then splitting at that node will result in two nodes with a higher average "node purity" than the original. So, we ensure continually better fits to the training data by continually splitting nodes.

So in this case the tree would split the data on **gender** first.

 © 2025 General Assembly

The Gini Impurity Index

The Gini index is actually pretty similar to its cousin **cross-entropy** and shares its key benefit: differentiability.

However in practice the Gini index is preferred because of its preference for pure nodes.

 © 2025 General Assembly

Decision Trees

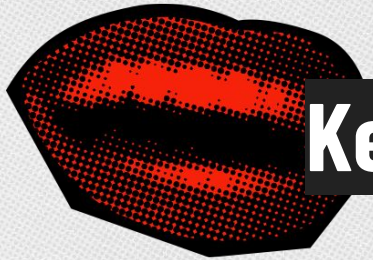
Let's see it for ourselves!

- ☐ Open **02-decision-trees.ipynb**



 © 2025 General Assembly

Bring It Home



Key takeaways



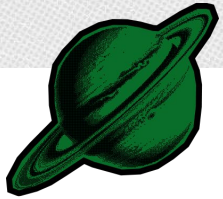
Decision Trees are very interpretable

Easy to describe to stakeholders



They are also useful

They're nonparametric, the scale of features doesn't matter, and they’re fst models to fit!



But they are prone to overfitting

And models tend to be overly simple when tuning, work poorly with imbalanced data

 © 2025 General Assembly


GA
GENERAL ASSEMBLY